VIETNAM NATIONAL UNIVERSITY, HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



DEEP LEARNING AND ITS APPLICATIONS

CO3133 – SEMESTER-261

ASSIGNMENT 2 – M2 DRAFT

Video-Disjoint Semantic Segmentation of Marine Debris in TrashCan

Completed U-Net evaluation and partial SegFormer-B0 training

Class: CC01

Instructor: Dr. Le Thanh Sach

Group: Tiki-taka

Milestone: M2 Draft

Evidence cutoff: 28 October 2026

No.	Group member	Student ID
1	Nguyen Anh Khoa	2452539
2	Nguyen Hao Thien	2453194
3	Ta Tuan Khai	2452515

Ho Chi Minh City, 28 October 2026

Ho Chi Minh City University of Technology
Faculty of Computer Science and Engineering



Abstract

The project has completed a scratch U-Net experiment on a video-disjoint TrashCan-Material split and has progressed to pretrained SegFormer-B0: S0 has 15 completed main-training epochs, while S1 has completed its GPU pilot only.

The validation-selected U-Net checkpoint obtains test foreground mean intersection over union (mIoU) 0.035627 and foreground Dice 0.058519. The partial S0 run reaches best-so-far validation foreground mIoU 0.195336 at epoch 15; this is a validation progress result, not a SegFormer test result or a final model ranking.

The next stage is to complete S0, train the paired S1 configuration with color augmentation, and compare the two completed runs using the fixed validation rule before interpreting their test results. The current evidence does not establish the effect of augmentation or variability across seeds.

Keywords: semantic segmentation; marine debris; TrashCan; U-Net; SegFormer; video-disjoint evaluation; progress report.

Table of Contents

Abstract **i**

1 Problem and data description **1**
1.1 Motivation and scope of this draft 1
1.2 Dataset source and provenance 1
1.3 Video-disjoint partition and class support 2
1.4 Mask conversion, preprocessing and checks 4

2 Methodology **6**
2.1 Implemented pipeline 6
2.2 Scratch U-Net baseline 6
2.3 Pretrained SegFormer-B0 and full fine-tuning 6
2.4 Objective and evaluation metrics 7
2.5 Training budget, checkpointing and environment 8
2.6 Controlled factor and current execution status 8

3 Implementation results at the current snapshot **10**
3.1 Completed U-Net training and evaluation 10
3.2 Measured SegFormer pilots 11
3.3 S0 main-training progress: 15 completed epochs 12
3.4 Per-class overlap and error patterns 13
3.5 Analytical background-only reference 15
3.6 Compute records and qualitative evidence boundary 16
3.7 S1 status and the pending augmentation comparison 16

4 Limitations and next experiments **17**

5 Conclusion **18**

6 Reproducibility and evidence record **18**
6.1 Source, effective configuration and provenance 18
6.2 Resume and artifact verification scope 19

7 Academic integrity and AI-use disclosure **19**

1 Problem and data description

1.1 Motivation and scope of this draft

Underwater debris is difficult to distinguish from marine life and the seafloor because object appearance, scale, illumination and occlusion vary. TrashCan supplies underwater images with object-level annotations for marine debris research[1]. This project converts the polygons into a semantic segmentation task: each valid image pixel receives one class label, without distinguishing separate instances of the same class.

For an RGB image $x \in \mathbb{R}^{H\times W\times 3}$, the model predicts a map $\hat{y} \in \{0,\dots,16\}^{H\times W}$. There are **17 output classes: background plus 16 foreground categories**. The foreground comprises ROV, plant, six animal categories and eight trash categories. Ignore label 255 is excluded from supervision and evaluation.

The research question is whether a pretrained modern segmentation model and modest photometric augmentation improve foreground overlap under a split that separates videos. This M2 draft reports the work completed through 28 October 2026: a completed U-Net experiment, an incomplete S0 SegFormer main run, and successful S0/S1 GPU pilots. It separates those observations from the remaining controlled comparison.

1.2 Dataset source and provenance

The working dataset is TrashCan 1.0 Material. The exploratory data analysis (EDA), measured on 5 October 2026, records 7,212 RGB images,12,336 polygon annotations and 312 filename-derived video groups. There are 3,245 images at 480×360 and 3,967 at 480×270. Source imagery is credited to JAMSTEC and the dataset to Hong, Fulton and Sattar[1].

The source archive's license file and the annotation JSON license records are retained with the project evidence. The JSON records identify an Attribution-NonCommercial-ShareAlike entry; the archive also contains its own attribution and use statement. The report preserves both records rather than replacing them with an inferred license. Raw images, checkpoints and the large dataset archive are not redistributed in the report source package.

The instructor approved the proposal according to the project owner's confirmation on 26 October 2026. This statement records the confirmation used by the training setup; it is not an independently inspected learning-management-system submission record.

1.3 Video-disjoint partition and class support

The original source partitions contain 6,008 training images and 1,204 validation images. They share 127 filename-derived video IDs. Separating individual frames using those source labels would therefore allow frames from the same video into different evaluation partitions.

The project instead uses a fixed seed 36 manifest that assigns each video to exactly one partition. Metadata-based split construction uses deterministic greedy/local-search candidates, with at least three supporting training videos and two supporting videos in each held-out partition per foreground class. Table 1.1 records the resulting image, video and annotation counts. Pairwise video-ID overlap is zero.

Table 1.1: Fixed video-disjoint partition, split seed 36. Native mask percentages are retained from the 5 October 2026 EDA.

Partition	Images	Videos	Annotations	Background (%)	Ignore (%)
Train	5,057	207	8,563	89.13	0.0371
Validation	1,077	54	1,956	89.34	0.0754
Test	1,078	51	1,817	90.33	0.0238

Video grouping addresses frame-level overlap under the chosen ID convention. It does not prove independence of dive locations or visually related scenes carrying different video IDs.

Class imbalance remains substantial after splitting. Table 1.2 gives each foreground class's annotation count, video support and native supervised pixel totals. The small support for categories such as animal shells makes macro overlap and per-class results more informative than background-dominated accuracy.

Table 1.2: Foreground class support in the fixed split. Pixel counts are native-resolution supervised pixels in millions, before letterboxing.

Class	Ann.	Videos (train/val/test)			Pixels, M (train/val/test)		
rov	3317	99	22	24	39.554	7.164	6.736
plant	507	17	10	5	2.363	0.586	0.661
animal_fish	764	33	7	5	4.421	1.444	1.577
animal_starfish	398	9	3	3	0.123	0.037	0.022
animal_shells	249	3	2	2	0.104	0.087	0.006
animal_crab	309	9	3	3	0.789	0.199	0.196
animal_eel	343	24	6	5	0.255	0.057	0.076
animal_etc	235	12	6	7	0.805	0.079	0.177
trash_etc	2039	107	23	23	9.145	1.921	1.601
trash_fabric	314	14	4	3	2.285	0.422	0.352
trash_fishing_gear	196	10	2	2	1.465	0.328	0.215
trash_metal	1126	42	12	8	3.590	0.638	1.275
trash_paper	193	6	2	2	0.947	0.257	0.056
trash_plastic	1864	61	16	20	12.865	2.535	2.480
trash_rubber	142	3	2	2	0.203	0.243	0.047
trash_wood	340	14	3	3	2.900	0.467	0.332

These counts describe dense native masks before resizing; they must not be mixed with the 512-canvas pixel totals reported by the evaluator.

The retained EDA figures provide a visual summary of the same imbalance and group coverage. Figure 1.1 shows category frequencies, and Figure 1.2 shows support across video groups. Counts are from the 5 October audit rather than a new raw-data measurement in this report update.

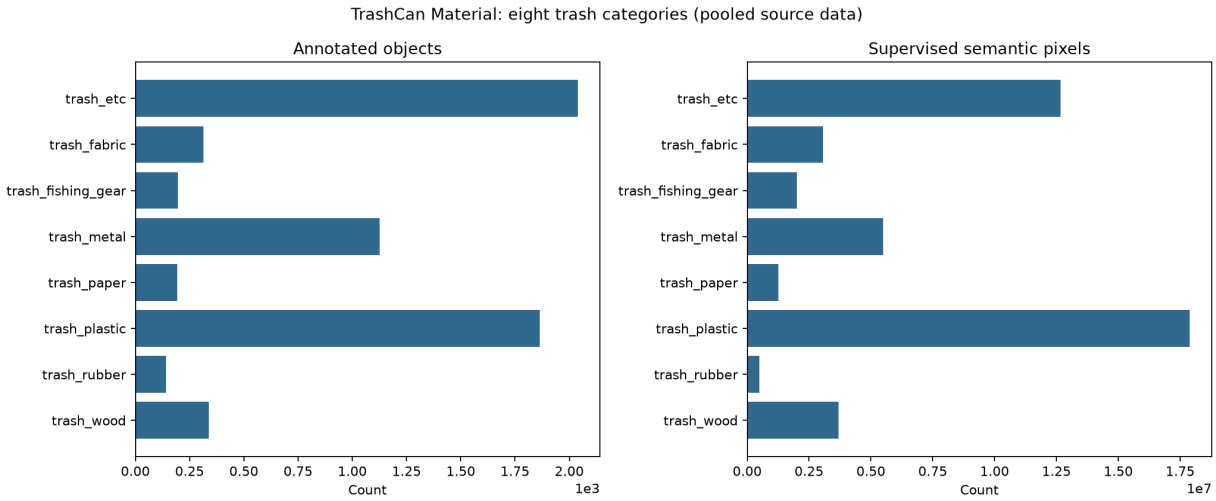


Figure 1.1: Material-category distribution from the retained EDA.

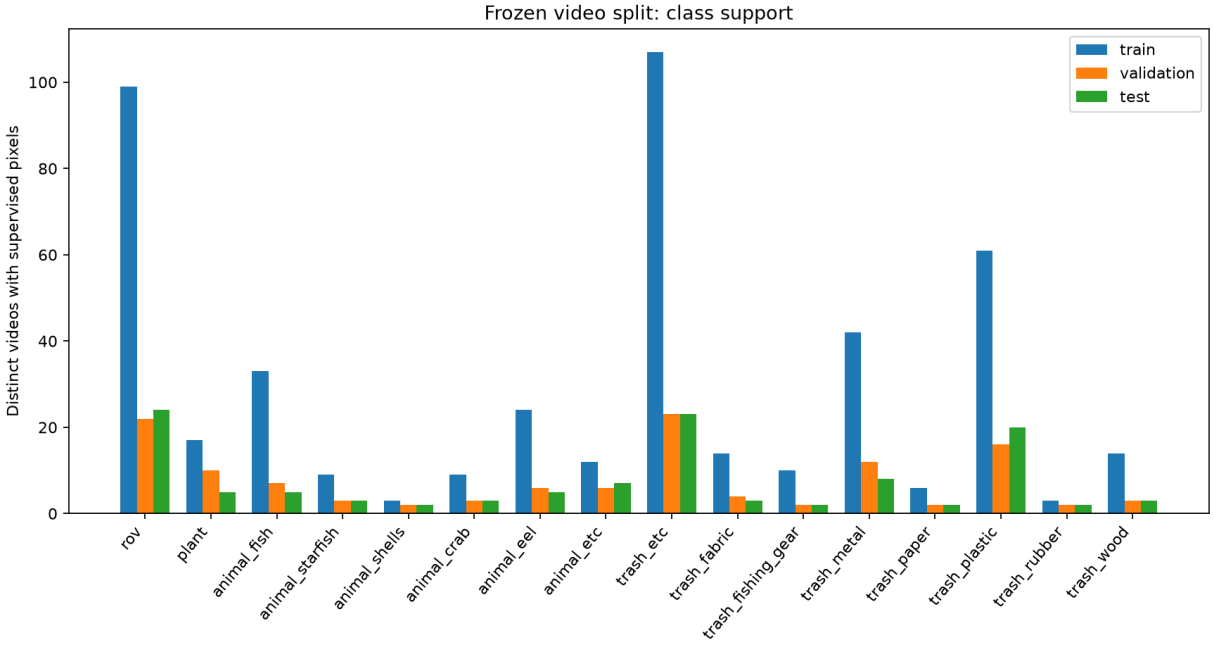


Figure 1.2: Per-class video support under the fixed seed 36 split.

The split provides class coverage in every partition while retaining rare classes; it does not remove their statistical uncertainty.

1.4 Mask conversion, preprocessing and checks

Polygons are rasterized on the native image canvas with Pillow 12.3.0. Same-class polygons are unioned; pixels where different foreground classes overlap receive ignore 255. Out-of-

canvas regions are clipped. The EDA found no annotation that rasterized to an empty mask under this converter.

Each image is letterboxed to 512×512 while preserving aspect ratio. RGB uses bilinear resizing, masks use nearest-neighbor resizing, RGB padding is black and target padding is ignore 255. The tensor is normalized with ImageNet channel means $(0.485, 0.456, 0.406)$ and standard deviations $(0.229, 0.224, 0.225)$. There is no label reduction or second image-processor resize.

The training adapter checks the published manifest digest, source annotation hashes, image dimensions, membership and native target hash. The supplied checks cover 5,057 training and 1,077 validation images. Figure 1.3 reproduces existing EDA annotation checks; it is not a gallery of model predictions. At evaluation resolution, ignored pixels include the much larger letterbox padding region as well as native annotation conflicts. For example, U-Net test ignore fraction is 34.06%; this does not mean that 34.06% of the native annotations are conflicting.

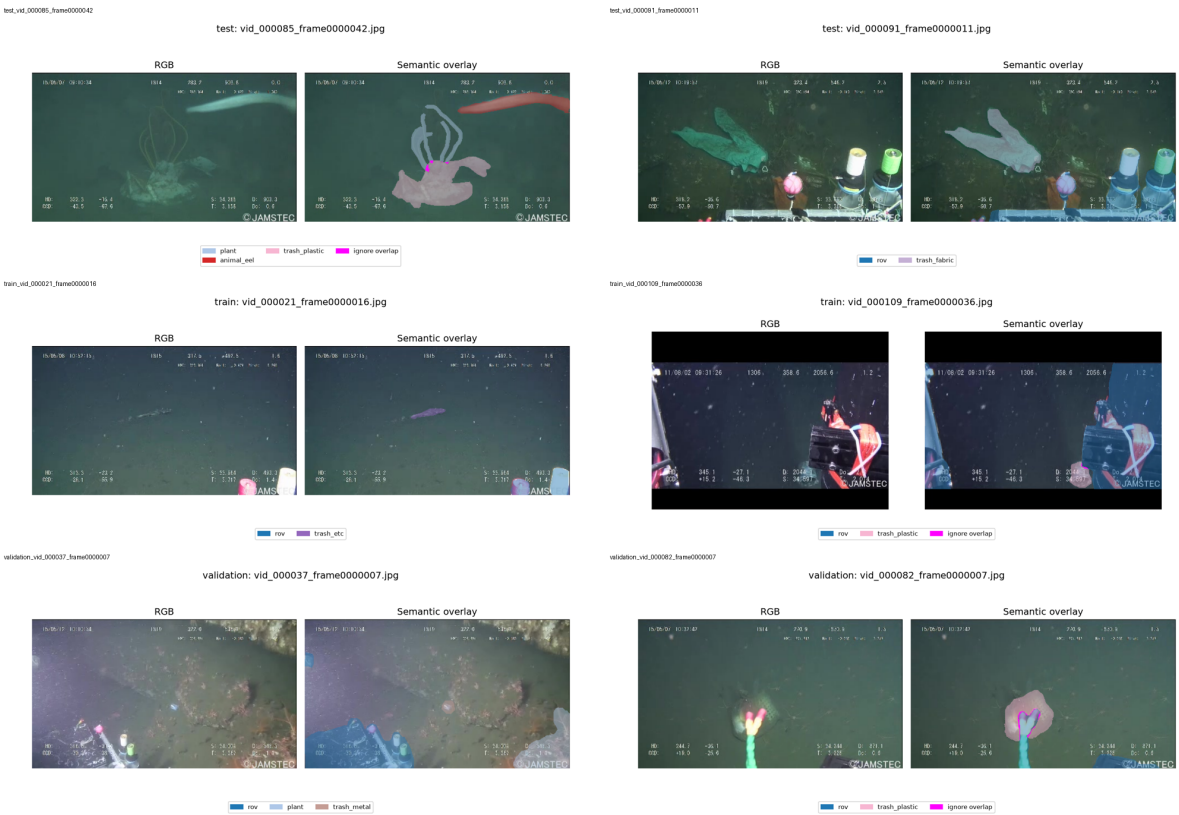


Figure 1.3: Six retained native annotation-QA examples, two per partition. RGB/mask overlays check conversion and alignment; they are ground truth, not inference outputs.

2 Methodology

2.1 Implemented pipeline

The common pipeline keeps data conversion, objective and scoring consistent across models. Figure 2.1 shows the main flow; test data enters only the final evaluation stage of a completed run.

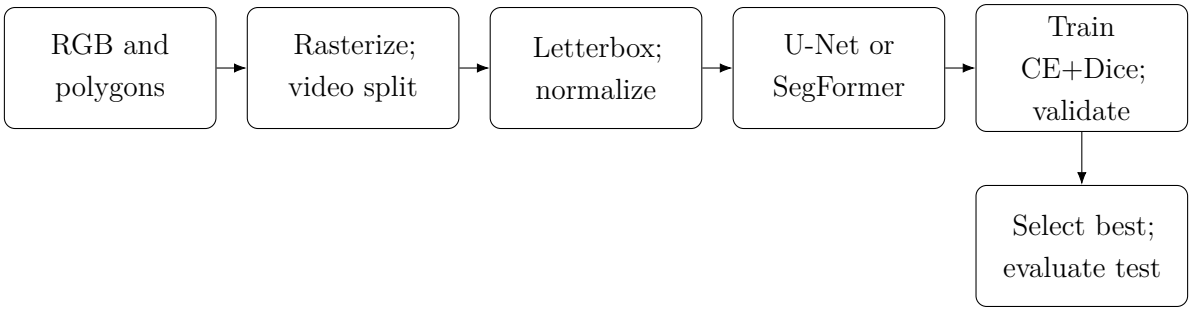


Figure 2.1: Shared segmentation workflow. Test scores do not select the training epoch.

This separation makes the S0/S1 experiment a change to training RGB color augmentation while preserving the mask and evaluation conventions.

2.2 Scratch U-Net baseline

The compact U-Net follows an encoder–decoder design with skip connections[2]. The implemented version has four downsampling stages and channel widths 16,32,64,128,256. Each block contains two bias-free 3×3 convolutions, eight-group Group Normalization and ReLU. Downsampling uses 2×2 max pooling. The decoder upsamples bilinearly with `align_corners=false`, concatenates encoder features, and applies another two-convolution block. A 1×1 head produces 17 logits per pixel.
All 1,964,369 parameter elements are trainable, with zero frozen parameters. The model is initialized from scratch. Its completed run supplies the initial architecture baseline and the only model test result in the current snapshot.

2.3 Pretrained SegFormer-B0 and full fine-tuning

SegFormer combines a hierarchical Transformer encoder with a lightweight multilayer-perceptron decoder[3]. The implementation loads `nvidia/segformer-b0-finetuned-ade-512-512`, pinned to revision `489d5cd81a0b59fab9b7ea758d3548ebe99677da`. The pretrained encoder and compatible decoder weights are retained, while the 150-class ADE20K classifier is replaced by a newly initialized 17-class classifier.

The complete model is fine-tuned end to end: encoder, decoder and classifier parameters all receive updates. Total and trainable counts are both 3,718,513; frozen count is zero. Pretrained initialization therefore does not imply a frozen backbone. Lower-resolution raw logits are bilinearly interpolated to the 512-canvas target before computing loss and argmax predictions.

S0 and S1 are initialized independently from the same checkpoint and seed 69420. Only S0 currently has main-run metadata, so equality of the two actual main initial-state hashes cannot yet be established from this snapshot. The setup checks this equality once both main-run records exist; the earlier synthetic authoring check is separate from that future run-level check.

2.4 Objective and evaluation metrics

Training minimizes the sum of multiclass cross-entropy and foreground soft Dice loss, each with coefficient 1:

$$\mathcal{L} = \mathcal{L}_{\mathrm{CE}} + \frac{1}{|C_B|} \sum_{c \in C_B} \left(1 - \frac{2 \sum_{i \in V_B} p_{ic} y_{ic} + \epsilon}{\sum_{i \in V_B} (p_{ic} + y_{ic}) + \epsilon} \right). \tag{2.1}$$

Here V_B is the set of valid pixels in a batch, C_B contains foreground classes with target support in that batch, p_{ic} is the softmax probability, y_{ic} is the one-hot target and $\epsilon = 10^{-6}$. Cross-entropy ignores 255. A batch with no foreground contributes zero Dice loss; an all-ignore batch is skipped for optimization. Logged epoch loss is a valid-pixel-weighted average of batch objectives, not a Dice score calculated from the epoch confusion matrix. For hard predictions, the evaluator accumulates one 17-by 17 confusion matrix across the whole partition. For class c,

$$\mathrm{IoU}_c = \frac{TP_c}{TP_c + FP_c + FN_c}, \qquad \mathrm{Dice}_c = \frac{2TP_c}{2TP_c + FP_c + FN_c}. \tag{2.2}$$

Rows are ground truth and columns are predictions. Foreground mIoU/Dice average IDs 1–16; trash mIoU averages IDs 9–16. All-class metrics include background. Undefined denominators are omitted and the number of defined classes is recorded. All 16 foreground classes have defined IoU in the reported held-out scores, including classes with zero intersection. Pixel accuracy divides the confusion diagonal by all valid pixels.

2.5 Training budget, checkpointing and environment

Both model families use AdamW, weight decay 0.01, batch 4, up to 40 epochs, patience 8, FP 32, no learning-rate scheduler and no mixed precision. U-Net learning rate is 10^{-3}; SegFormer uses 6×10^{-5}. Training seed 69420 is distinct from split seed 36. The training loader shuffles with a separate generator seeded by run seed plus epoch; validation/test are unshuffled.

The best checkpoint maximizes validation foreground mIoU; a tie uses lower validation loss, then the earlier epoch. Each completed epoch saves model, AdamW state, random-number states, history, best selection and early-stopping counter. Resume restores this state; an incomplete epoch is repeated. The U-Net continuation belongs to the same experiment and is not an independent seed.

The supplied runs/pilots record a Tesla T4 with 14.56 GiB memory, Python 3.13.16, PyTorch 2.11.0+cu 130, torchvision 0.26.0+cu 130 and NumPy 2.1.3. Pillow 12.3.0 and Transformers 4.57.6 are pinned. TF 32 is disabled and CUDA deterministic algorithms use warning-only mode for the bilinear-backward limitation. These settings and saved RNG state do not promise cross-runtime GPU bitwise identity.

2.6 Controlled factor and current execution status

S0 disables color jitter. S1 applies it to training RGB with probability 0.5: brightness, contrast and saturation factors are sampled in $[0.9, 1.1]$, hue shift in $[-0.01, 0.01]$, with shuffled operation order. Masks are unchanged. A seed/epoch/image-specific random stream keeps augmentation separate from model and sampling randomness. There is no geometric augmentation; held-out images are unaugmented.

The paired setup locks both configurations and their numerical environment before main training or test evaluation. S0/S1 share architecture, pretrained revision, seed, split, normalization, optimizer, learning rate, loss and budget. Run name and train color switch are their only treatment differences. Table 2.1 distinguishes planned settings from completed execution.

Table 2.1: Experiment status at the supplied 28 October snapshot.

Configuration	Completed evidence	Status of remaining work
U: scratch U-Net	23 epochs, early stopping; best 15; validation and test scored	Main/test complete for seed 69420
S0: pretrained B0, color off	20-step pilot;15 main epochs; best-so-far 15	Main incomplete; no test result
S1: pretrained B0, color on	20-step pilot; frozen configuration	Main not started in supplied snapshot
Constant background	Metrics derived analytically from held-out supports	No trainable model or inference run

The S0/S1 hypothesis is that mild color changes improve held-out foreground overlap. It cannot yet be tested because S1 has no main-training result. U versus S0 is a descriptive comparison of model configurations: architecture, initialization and learning rate change together, and the current training completion states differ. It does not isolate the causal effect of pretraining.

3 Implementation results at the current snapshot

3.1 Completed U-Net training and evaluation

The U-Net experiment completed 23 epochs and stopped after eight consecutive epochs without an improved validation foreground mIoU. Its selected checkpoint is epoch 15. This includes an epoch-boundary continuation from epoch 7 after host kernel/RAM descriptors changed; the continuation retains the same experiment, seed and budget.

Figure 3.1 shows training loss decreasing from 1.5083 at epoch 1 to 0.6102 at epoch 23 while training foreground mIoU rises from 0.0132 to 0.4072. Validation foreground mIoU peaks at 0.053085 at epoch 15; its final value is 0.047450. Validation loss is lowest at epoch 5 (1.271954) and ends at 1.516843. The growing train–validation gap is consistent with overfitting in this run, without establishing a single cause.

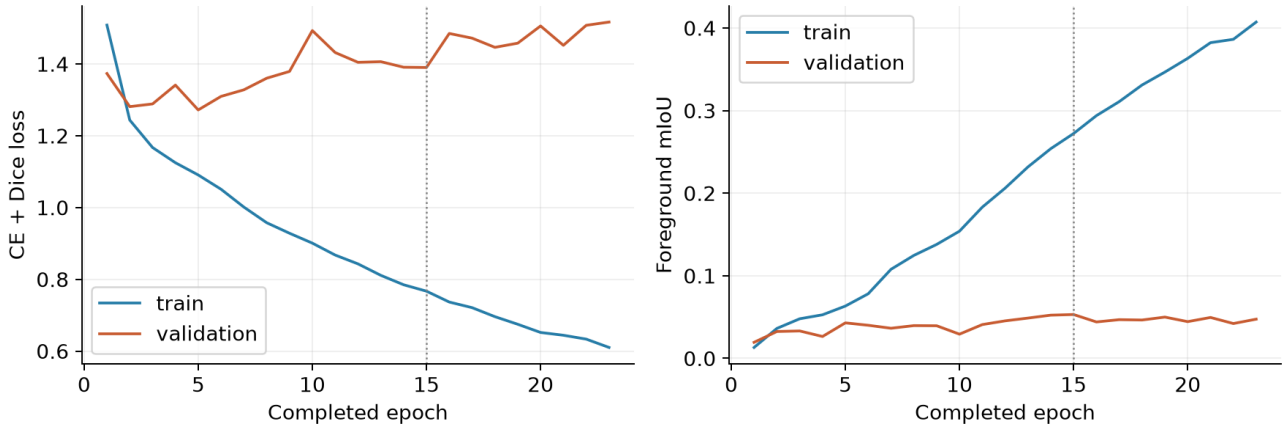


Figure 3.1: Completed U-Net history. The dotted line marks the validation-selected epoch 15; the run ends at epoch 23.

The metric used to choose the checkpoint is foreground mIoU, so minimum validation loss and selected epoch need not coincide.

The selected checkpoint was replayed on validation and evaluated on all 1,078 test images. Table 3.1 places these completed U-Net scores next to S0's best-so-far validation score, while retaining the different execution states.

Table 3.1: Measured metrics at epoch 15. U-Net is a completed run; S0 is a15-epoch partial snapshot. All scores are on a0–1 scale.

Metric	U validation	U test	S0 validation
Foreground mIoU	0.053085	0.035627	0.195336
Foreground Dice	0.084369	0.058519	0.280506
Trash mIoU	0.032704	0.014652	0.137865
All-class mIoU	0.103646	0.087050	0.239732
All-class Dice	0.135542	0.111123	0.321323
CE + Dice loss	1.390246	1.407840	1.134105
Valid pixels (M)	176.765	186.340	176.765
Ignored pixels (%)	37.39	34.06	37.39

U-Net test foreground mIoU is 0.035627, foreground Dice 0.058519 and trash mIoU 0.014652. Its validation-to-test foreground mIoU difference is 0.017458. These are observations for a single seed and fixed split, rather than estimates of variability across repeated runs.

3.2 Measured SegFormer pilots

Both SegFormer arms completed 20 optimization steps on a fixed real training batch before their main configurations were frozen. Table 3.2 records finite-loss, gradient and capacity diagnostics. Each pilot's final loss is lower than its initial loss, and peak reserved CUDA memory remains below 90% of GPU memory.

Table 3.2: Measured 20-step, fixed-train-batch diagnostics on the T4. These are capacity/gradient checks, not held-out segmentation results.

Arm	First loss	Last loss	Peak reserved (GiB)	GPU memory (GiB)
S0	3.8140	3.0349	2.70	14.56
S1	3.8145	3.0304	2.73	14.56

Pilot models are discarded; main training starts again from the prescribed pretrained checkpoint and seed. The similar pilot losses do not measure the effect of augmentation on held-out segmentation. Pilot memory is also not a measurement of peak memory across the complete main-training workload.

3.3 S0 main-training progress: 15 completed epochs

The supplied S0 checkpoint/history contains 15 completed epochs, below the configured 40-epoch limit. Epoch 15 is currently best, and the early-stopping counter is zero. Therefore the snapshot is **an incomplete main run**, not a completed 40-epoch or early-stopped SegFormer experiment.

Figure 3.2 shows the available S0 history. Training loss falls from 1.250555 at epoch 1 to 0.247658 at epoch 15, and training foreground mIoU rises from 0.084281 to 0.744726. Validation foreground mIoU rises from 0.100556 to its current maximum 0.195336. At epoch 15, validation Dice is 0.280506, trash mIoU 0.137865 and loss 1.134105.

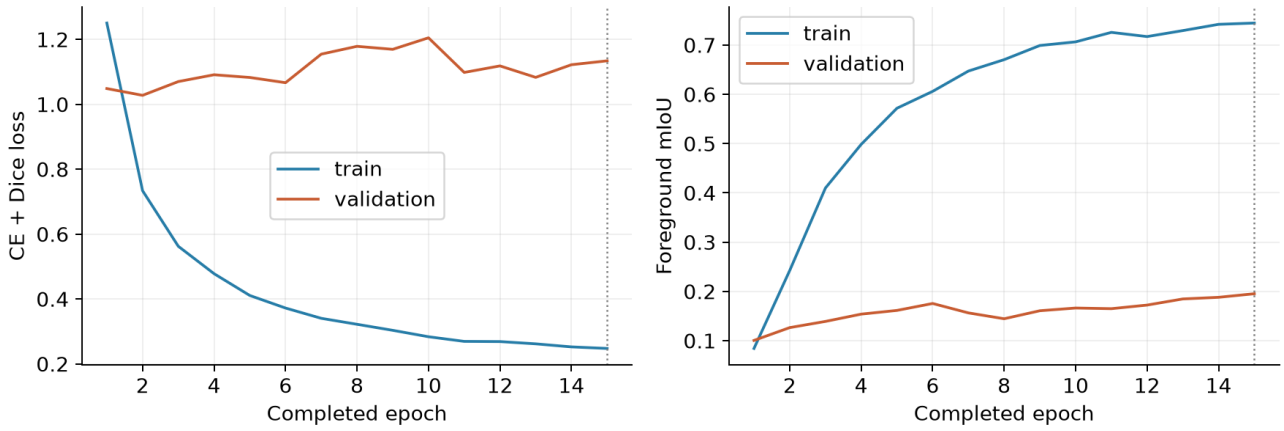


Figure 3.2: Partial SegFormer S0 history through epoch 15. The selected epoch is best-so-far; training has not reached its completion criterion.

The latest train–validation foreground mIoU gap is 0.549390. Validation loss reached its lowest observed value at epoch 2 (1.027752), whereas foreground mIoU is highest at epoch 15. Better hard-mask overlap and lower probabilistic loss need not occur together; the saved foreground-mIoU selection rule remains unchanged. The large training/validation gap motivates continued scrutiny of class generalization, rather than treating the training score as held-out quality.

The partial S0 validation mIoU exceeds the completed U-Net validation mIoU by 0.142251 (14.2251 percentage points). This comparison uses the same validation membership and scoring convention. It is evidence that the current S0 configuration learns useful foreground structure; architecture, initialization, learning rate and execution progress differ, so it is neither a final ranking nor a controlled estimate of pretraining's contribution. No S0 test score or independent best-checkpoint validation replay is included in the supplied snapshot; the S0 values come from completed epoch validation in the training history.

3.4 Per-class overlap and error patterns

Table 3.3 preserves each class's measured overlap, and Figure 3.3 compares only the two validation columns. The U test column remains a separate completed held-out evaluation.

Table 3.3: Class-wise overlap. S0 columns use its best-so-far validation checkpoint; no S0 test result is claimed.

ID	Class	U val IoU	U test IoU	S0 val IoU	S0 val Dice
0	background	0.9126	0.9098	0.9501	0.9744
1	rov	0.4722	0.3564	0.6907	0.8171
2	plant	0.0190	0.0038	0.1074	0.1940
3	animal_fish	0.0162	0.0367	0.5031	0.6694
4	animal_starfish	0.0000	0.0000	0.0323	0.0627
5	animal_shells	0.0000	0.0000	0.0000	0.0000
6	animal_crab	0.0804	0.0292	0.2820	0.4399
7	animal_eel	0.0000	0.0268	0.4068	0.5784
8	animal_etc	0.0000	0.0000	0.0000	0.0000
9	trash_etc	0.0802	0.0325	0.0575	0.1088
10	trash_fabric	0.0358	0.0145	0.0729	0.1359
11	trash_fishing_gear	0.0000	0.0000	0.0078	0.0155
12	trash_metal	0.0390	0.0036	0.0618	0.1163
13	trash_paper	0.0000	0.0000	0.1849	0.3121
14	trash_plastic	0.0901	0.0666	0.2474	0.3966
15	trash_rubber	0.0000	0.0000	0.0012	0.0024
16	trash_wood	0.0166	0.0000	0.4694	0.6389

S0's strongest foreground validation IoUs at epoch 15 are ROV 0.6907, animal fish 0.5031, trash wood 0.4694 and animal eel 0.4068. Animal shells and animal other still have zero validation IoU. Improvement is not uniform: trash other has S0 validation IoU 0.0575 compared with U-Net 0.0802. A higher macro score therefore does not imply every debris category has improved.

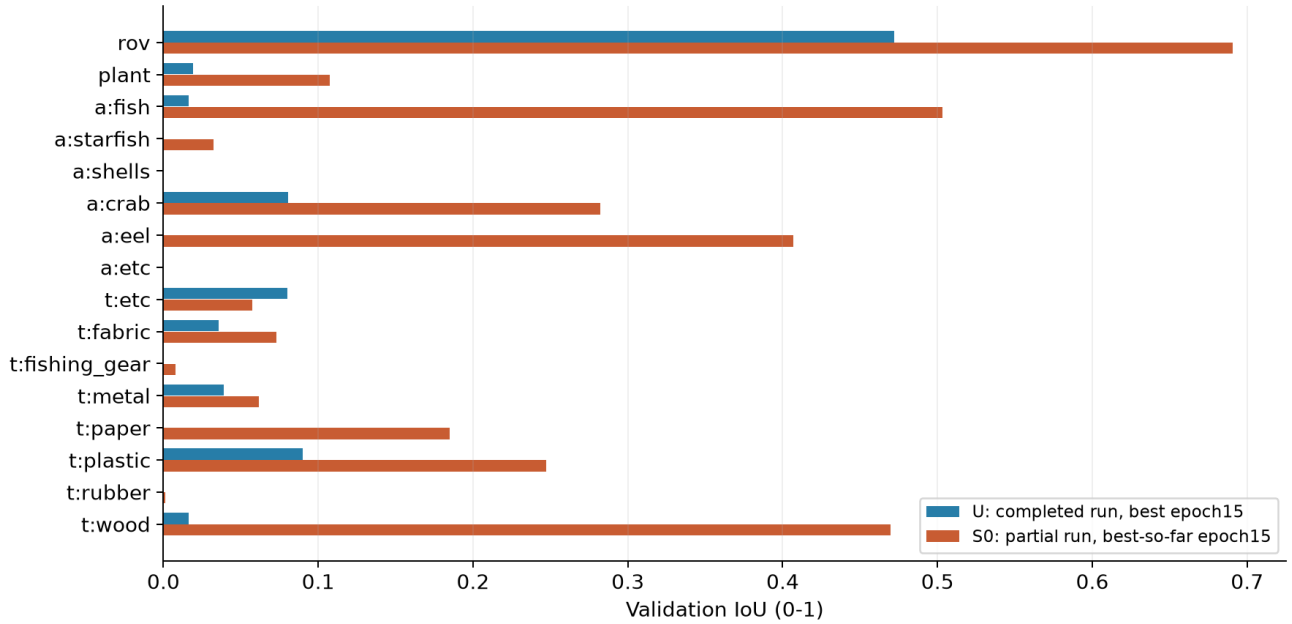


Figure 3.3: Validation IoU by foreground class. U is a completed run and S0 is partial; both columns refer to epoch 15. Prefixes a:/t: denote animal/trash.

The U-Net test result remains weak across many categories: six foreground classes have zero IoU, and its strongest foreground class is ROV (0.3564). Sparse support is a plausible contributor, but aggregate counts do not distinguish class imbalance, appearance shift, boundary errors and optimization effects as causes.

To show how these errors are distributed, Figure 3.4 normalizes each confusion row by its true-class support. This puts frequent and rare categories on a common within-class scale. The left panel is U test; the right is S0 validation, so the panels are not a same-partition model comparison.

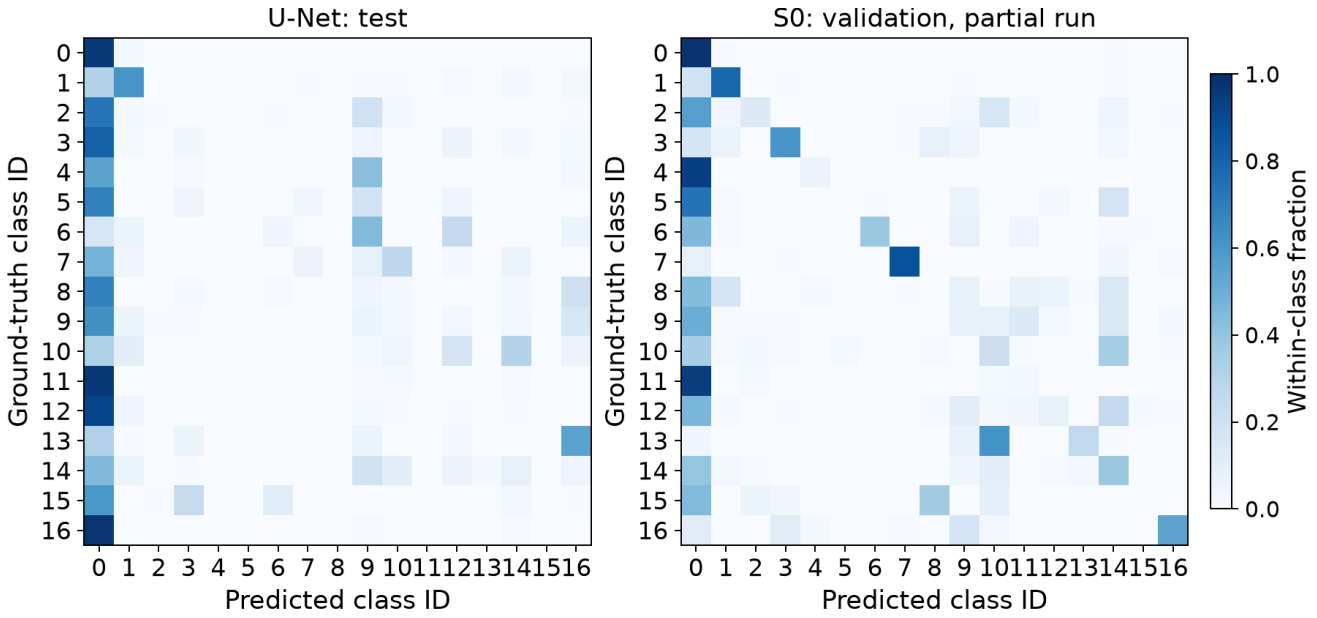


Figure 3.4: Row-normalized confusion: U-Net test (left) and partial S0 validation (right). Both color scales run 0–1; class IDs are defined in Table 3.3.

In the U test matrix,9,174,492 of 17,988,134 foreground pixels (51.00%) are assigned to background. This directly measures foreground omission. It does not identify whether the missed pixels correspond to small objects, occlusion or particular visual conditions; that requires prediction-level image inspection.

3.5 Analytical background-only reference

A predictor that assigns every valid pixel to background provides a useful check on the interpretation of accuracy. Its confusion matrix can be constructed without inference: place each true-class support in column 0. Table 3.4 derives this reference from the already saved held-out confusion-row totals; it is an analytical calculation on existing labels, not a newly trained model or an additional checkpoint evaluation.

Table 3.4: Analytical constant-background reference derived from existing confusion-row supports; no model inference is required.

Partition / predictor	Pixel acc.	FG mIoU	FG Dice	All mIoU
Validation / background	0.894015	0.000000	0.000000	0.052589
Test / background	0.903466	0.000000	0.000000	0.053145
Test / U-Net	0.894205	0.035627	0.058519	0.087050

On test, constant background obtains pixel accuracy 0.903466, above U-Net's 0.894205,

yet has foreground mIoU and Dice exactly zero. U-Net's nonzero foreground overlap and higher all-class mIoU show a different trade-off. This establishes why the study uses foreground macro metrics as primary outcomes: pixel accuracy alone can favor a predictor that never finds debris or marine-life classes.

3.6 Compute records and qualitative evidence boundary

S0's supplied progress log contains 15 training-phase records totaling 14,243.52 seconds (3.9565 hours) and 15 validation records totaling 838.61 seconds (13.98 minutes). These are phase durations for the 15 completed epochs, excluding setup, pilots, checkpoint writes, idle/disconnected time and final test processing. They are not complete-experiment wall time or inference latency. No inference benchmark is reported, and a completed U/S0 cost comparison is deferred until both scopes can be reconciled.

The current report includes annotation-QA overlays and aggregate confusion evidence. It does not present ground-truth overlays as model predictions. Prediction galleries, including foreground misses and inter-class confusions, will be exported from selected checkpoints after the respective run completes; no new checkpoint inference was performed during this report update.

3.7 S1 status and the pending augmentation comparison

S1 has a successful GPU pilot and a frozen color-on configuration, but the supplied S1 folder contains no main-run directory, epoch history or main checkpoint. Consequently there is no S1 validation/test score to compare with S0, no main-run initial-state hash pair to verify yet, and no observed augmentation effect.

The experiment remains defined as S0 versus S1 with the same architecture, initialization scheme, seed, split, optimizer, learning rate and budget. Completing this pair is the next experiment milestone; the 20-step pilot is not substituted for that comparison.

4 Limitations and next experiments

The evidence covers one completed U-Net run and one partial SegFormer S0 run, each at seed 69420. SegFormer test performance, the S0/S1 augmentation effect and multi-seed variability remain outside the measured results. The current S0 validation lead must not be interpreted as a generalization guarantee or a causal estimate of pretraining.
Video-disjoint membership prevents the measured 127 source video-ID overlaps, but filename-derived IDs do not establish independence of dives, locations or scenes. The semantic conversion ignores cross-class conflicts and discards instance identities. Letterboxing and class imbalance also affect what the reported pixel counts and overlap scores represent. The two zero-IoU S0 validation classes and the U-Net foreground misses remain substantive limitations despite improvements in other classes.
The next steps are sequenced to preserve the comparison:

1. Resume S0 from the saved epoch 15 state until the existing budget or early-stopping condition is reached. Retain the original settings and select best using validation foreground mIoU.

2. Train S1 from its prescribed pretrained initialization with only train color augmentation changed. Verify equality of the actual main initial-state hashes and compatibility of the numerical environments before interpreting the paired contrast.

3. Evaluate completed selected checkpoints with the existing fixed test stage, then report the two arms separately. Test results describe the completed configurations and must not guide changes to this pair.

4. Compare foreground mIoU/Dice and class-level changes; export prediction galleries to inspect foreground omission and rare-category failures. If compute permits, repeat the paired comparison with the prespecified additional seeds 67 and 69 to quantify seed variation.

5. Measure inference cost using the same device, input shape, precision, batch size and timing definition. Keep those measurements separate from epoch-phase and pilot timings.

If runtime resources are limited, executing S0 and S1 in separate sessions with the same pair tag and locked environment preserves the intended factor better than silently changing batch size or resolution in one arm. A deliberate protocol change requires a new pair and separately reported settings.

5 Conclusion

The project has moved beyond implementation setup. It has a verified video-disjoint data pipeline, a completed scratch U-Net training/test experiment, two successful SegFormer GPU pilots, and 15 completed epochs of pretrained SegFormer S0 main training.
U-Net selects epoch 15 after 23 completed epochs and reaches test foreground mIoU 0.035627 and Dice 0.058519. The incomplete S0 run reaches best-so-far validation mIoU 0.195336 at epoch 15, compared with U-Net validation 0.053085. The current S0 result is promising at the configuration level, while large train–validation gaps and persistent rare-class failures show that segmentation quality remains uneven.
This M2 draft establishes current progress, not a completed architecture or augmentation study. Completing S0 and S1, preserving validation-based selection, and then evaluating the selected checkpoints are required before answering whether the planned color augmentation helps. Robustness across seeds and inference-cost trade-offs remain subsequent measurements.

6 Reproducibility and evidence record

6.1 Source, effective configuration and provenance

The supplied U and S0 run records use source revision

`7df323698d1ae4d5122850a54eb38b01609d80f1`

and source digest

`13ea100cefed8e7bd13c38153ebd3321760804366e2b5a66b47d2076d7e0c503`.

The manifest canonical digest is

`87905f1a97dd2c04194e049ca61d1a7f3bbe2942e9fe17dc2d737c371cdce943`.

Canonical manifest hashing excludes its own digest field. The selected U checkpoint SHA-256 is

`05aa0e8acf653eb56b4e3de979850e92687286e43a0cc75d65d63ba783f9cbe8`,

and the current best-so-far S0 checkpoint SHA-256 is

`9e6a881541693f81eba82f16c948160b393b082faad5d52685c7a26598cc862b`.

The source snapshots and these digests were checked against the supplied bytes during the 28 October report update. This verifies consistency of the archived artifacts; it is not an independent rerun on raw images.

The source repository's shared protocol file deliberately stores draft defaults. At execution, the notebook records a successful matching GPU pilot and the existing approval reference, freezes the run-local configuration, and saves it in the setup ledger, effective configuration and identity. The supplied frozen configurations agree with those records. Thus the draft defaults and frozen effective run settings describe different stages of the same setup procedure, rather than an unresolved contradiction.

The paired ledger also records source/helper hashes, base configurations and numerical-runtime settings. S0 and S1 frozen configs were checked to differ only in run name, color switch and their distinct pilot-evidence references. Separate pilot references record evidence; they are not an experimental treatment.

6.2 Resume and artifact verification scope

The supplied U continuation records a resume at epoch 7 with changes in kernel-release descriptor and total RAM. Its final history/checkpoint and test marker agree through epoch 23 and best 15. The original parent files are not part of that supplied continuation folder, so this report does not claim a new byte-by-byte verification against the absent parent checkpoint. The recovery implementation had separately passed CPU preservation checks when authored.

For S0, the audit verified checkpoint/metadata identities, selected epoch, full 15-epoch history, source-file hashes, configuration and all training/validation confusion-derived aggregate and per-class metrics. S1 verification covers the setup, pilot, manifest and helper bundle. The U validation/test outputs match the previously verified file hashes and confusion-derived metrics. All supplied input hashes remain unchanged.

The accompanying evidence files contain the input hash inventory, numerical verification JSON and CSV histories. Report tables and curves are generated from those audited records. No additional main training, checkpoint inference or test evaluation was run for report preparation.

7 Academic integrity and AI-use disclosure

OpenAI Codex assisted with the segmentation code/notebook implementation, checkpoint/provenance checks, report organization, prose and generation of tables/plots from saved artifacts. During this report update, scripts verified checkpoint metadata, recomputed metrics from existing confusion matrices and rendered the report; they did not train the models, run checkpoint inference or create annotation labels. The numerical training/evaluation outputs were produced by the project runs executed on Colab.

Representative report instructions were to write only through the current progress and distinguish completed U-Net, partial S0 and pilot-only S1. The authors retain responsibility for reviewing the data definitions, interpretations, citations and disclosed AI assistance before submission. No unverified contribution percentage is assigned in this draft.

References

[1] J. Hong, M. Fulton and J. Sattar. *TrashCan: A Semantically-Segmented Dataset towards Visual Detection of Marine Debris*.2020. https://arxiv.org/abs/2007.08097.

[2] O. Ronneberger, P. Fischer and T. Brox. *U-Net: Convolutional Networks for Biomedical Image Segmentation*.2015. https://arxiv.org/abs/1505.04597.

[3] E. Xie et al. *SegFormer: Simple and Efficient Design for Semantic Segmentation with Transformers*.2021. https://arxiv.org/abs/2105.15203.

[4] Tiki-taka. *TrashCan-Material EDA, fixed seed 36 manifest, source snapshots, setup/pilot ledgers and run artifacts*. EDA measured 5 October 2026; supplied training artifacts audited 28 October 2026. Machine-readable evidence accompanies this report source.